

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define int long long
5  #define endl '\n'
6  #define all(x) x.begin(), x.end()
7  #define rep(i, a, b) for (int i = a; i < b; i++)
8  #define pb push_back
9  #define mp make_pair
10 #define mod 1000000007
11
12 int n;
13 vector<int> v;
14
15 int main() {
16     cin >> n;
17     v.resize(n);
18     rep(i, 0, n) cin >> v[i];
19
20     sort(v.begin(), v.end());
21
22     int ans = 0;
23     rep(i, 0, n) {
24         int x = v[i];
25         int y = 0;
26         while (x > 0) {
27             y = x % 10;
28             x /= 10;
29         }
30         ans = max(ans, y);
31     }
32     cout << ans << endl;
33     return 0;
34 }

```

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <cmath>
5  #include <algorithm>
6  #include <fstream>
7  #include <ctime>
8  #include <cstdlib>
9
10 using namespace std;
11
12 const int MAX_GRID = 75;
13 int initial_grid[MAX_GRID][MAX_GRID];
14 int current_grid[MAX_GRID][MAX_GRID];
15 int goal_pattern[MAX_GRID][MAX_GRID];
16 bool is_fixed[MAX_GRID][MAX_GRID];
17 int board_size, init_blank_row, init_blank_col, blank_row, blank_col;
18
19 const int d_row[] = {-1, 1, 0, 0};
20 const int d_col[] = {0, 0, -1, 1};
21 const char action_labels[] = {'D', 'U', 'R', 'L'};
22
23 int bfs_visited[MAX_GRID][MAX_GRID];
24 int current_bfs_ticket = 0;
25 int trace_r[MAX_GRID][MAX_GRID];
26 int trace_c[MAX_GRID][MAX_GRID];
27 char trace_move[MAX_GRID][MAX_GRID];
28
```

1

```
void restore_board_state() {
    for (int i = 0; i < board_size; ++i) {
        for (int j = 0; j < board_size; ++j) {
            current_grid[i][j] = initial_grid[i][j];
            is_fixed[i][j] = false;
        }
    }
    blank_row = init_blank_row;
    blank_col = init_blank_col;
}
```

2

```
string clean_up_redundant_moves(string raw_moves) {
    bool is_modified = true;
    while (is_modified) {
        is_modified = false;
        string optimized = "";
        for (char action : raw_moves) {
            if (!optimized.empty()) {
                char prev_action = optimized.back();
                if ((prev_action == 'U' && action == 'D') || (prev_action == 'D' && action == 'U') ||
                    (prev_action == 'L' && action == 'R') || (prev_action == 'R' && action == 'L')) {
                    optimized.pop_back();
                    is_modified = true;
                    continue;
                }
            }
            optimized.push_back(action);
        }
        if (raw_moves.length() != optimized.length()) {
            raw_moves = optimized;
            is_modified = true;
        }
    }
    return raw_moves;
}
```

4

```
void apply_actions(const string& sequence) {
    for (char move : sequence) {
        int direction_idx = -1;
        for (int i = 0; i < 4; ++i) {
            if (action_labels[i] == move) direction_idx = i;
        }
        int next_r = blank_row + d_row[direction_idx];
        int next_c = blank_col + d_col[direction_idx];
        swap(current_grid[blank_row][blank_col], current_grid[next_r][next_c]);
        blank_row = next_r;
        blank_col = next_c;
    }
}
```

3

```
string route_blank_slot(int dest_r, int dest_c, int block_r, int block_c) {
    if (blank_row == dest_r && blank_col == dest_c) return "";

    current_bfs_ticket++;
    int q_front = 0, q_back = 0;
    int q_row_arr[10000], q_col_arr[10000];

    q_row_arr[q_back] = blank_row;
    q_col_arr[q_back] = blank_col;
    q_back++;
    bfs_visited[blank_row][blank_col] = current_bfs_ticket;

    bool target_reached = false;

    while (q_front < q_back) {
        int r = q_row_arr[q_front], c = q_col_arr[q_front];
        q_front++;

        if (r == dest_r && c == dest_c) {
            target_reached = true;
            break;
        }

        for (int i = 0; i < 4; ++i) {
            int next_r = r + d_row[i], next_c = c + d_col[i];
            if (next_r >= 0 && next_r < board_size && next_c >= 0 && next_c < board_size) {
                if (is_fixed[next_r][next_c]) continue;
                if (next_r == block_r && next_c == block_c) continue;

                if (bfs_visited[next_r][next_c] != current_bfs_ticket) {
                    bfs_visited[next_r][next_c] = current_bfs_ticket;
                    trace_r[next_r][next_c] = r;
                    trace_c[next_r][next_c] = c;
                    trace_move[next_r][next_c] = action_labels[i];
                    q_row_arr[q_back] = next_r;
                    q_col_arr[q_back] = next_c;
                    q_back++;
                }
            }
        }
    }

    string final_path = "";
    if (!target_reached) return final_path;

    int temp_r = dest_r, temp_c = dest_c;
    while (temp_r != blank_row || temp_c != blank_col) {
        final_path += trace_move[temp_r][temp_c];
        int pr = trace_r[temp_r][temp_c];
        int pc = trace_c[temp_r][temp_c];
        temp_r = pr;
        temp_c = pc;
    }
    reverse(final_path.begin(), final_path.end());
    return final_path;
}
```

5

```
pair<int, int> calculate_tile_step(int start_r, int start_c, int end_r, int end_c) {
    current_bfs_ticket++;
    int q_front = 0, q_back = 0;
    int q_row_arr[10000], q_col_arr[10000];

    q_row_arr[q_back] = start_r;
    q_col_arr[q_back] = start_c;
    q_back++;
    bfs_visited[start_r][start_c] = current_bfs_ticket;

    bool target_reached = false;

    while (q_front < q_back) {
        int r = q_row_arr[q_front], c = q_col_arr[q_front];
        q_front++;

        if (r == end_r && c == end_c) {
            target_reached = true;
            break;
        }

        for (int i = 0; i < 4; ++i) {
            int next_r = r + d_row[i], next_c = c + d_col[i];
            if (next_r >= 0 && next_r < board_size && next_c >= 0 && next_c < board_size) {
                if (is_fixed[next_r][next_c]) continue;
                if (bfs_visited[next_r][next_c] != current_bfs_ticket) {
                    bfs_visited[next_r][next_c] = current_bfs_ticket;
                    trace_r[next_r][next_c] = r;
                    trace_c[next_r][next_c] = c;
                    q_row_arr[q_back] = next_r;
                    q_col_arr[q_back] = next_c;
                    q_back++;
                }
            }
        }

        int temp_r = end_r, temp_c = end_c;
        while (trace_r[temp_r][temp_c] != start_r || trace_c[temp_r][temp_c] != start_c) {
            int pr = trace_r[temp_r][temp_c];
            int pc = trace_c[temp_r][temp_c];
            temp_r = pr;
            temp_c = pc;
        }
        return {temp_r, temp_c};
    }
}
```

```

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    srand(time(NULL));

    cin >> board_size;

    for (int i = 0; i < board_size; ++i) {
        for (int j = 0; j < board_size; ++j) {
            cin >> initial_grid[i][j];
            if (initial_grid[i][j] == -1) {
                init_blank_row = i;
                init_blank_col = j;
            }
        }
    }

    for (int i = 0; i < board_size - 2; ++i) {
        for (int j = 0; j < board_size - 2; ++j) {
            cin >> goal_pattern[i][j];
        }
    }

    int global_minimum_moves = 2e9;

    cout << "=== Start Optimization (Monte Carlo) ===" << endl;

    for (int attempt = 1; ; ++attempt) {
        restore_board_state();
        string accumulated_moves = "";
        int path_cost[MAX_GRID][MAX_GRID];
        bool is_deadlocked = false;

        for (int target_r = 1; target_r <= board_size - 2; ++target_r) {
            for (int step = 1; step <= board_size - 2; ++step) {
                int target_c = (target_r % 2 != 0) ? step : (board_size - 1 - step);
                int expected_color = goal_pattern[target_r - 1][target_c - 1];
                for (int i = 0; i < board_size; ++i) {
                    for (int j = 0; j < board_size; ++j) {
                        path_cost[i][j] = 1e9;
                    }
                }

                current_bfs_ticket++;
                int q_front = 0, q_back = 0;
                int q_row_arr[10000], q_col_arr[10000];

                q_row_arr[q_back] = target_r;
                q_col_arr[q_back] = target_c;
                q_back++;

                path_cost[target_r][target_c] = 0;
                bfs_visited[target_r][target_c] = current_bfs_ticket;

                while (q_front < q_back) {
                    int curr_r = q_row_arr[q_front], curr_c = q_col_arr[q_front];
                    q_front++;
                    for (int i = 0; i < 4; ++i) {
                        int next_r = curr_r + d_row[i], next_c = curr_c + d_col[i];
                        if (next_r >= 0 && next_r < board_size && next_c >= 0 && next_c < board_size) {
                            if (!is_fixed[next_r][next_c] && bfs_visited[next_r][next_c] != current_bfs_ticket) {
                                bfs_visited[next_r][next_c] = current_bfs_ticket;
                                path_cost[next_r][next_c] = path_cost[curr_r][curr_c] + 1;
                                q_row_arr[q_back] = next_r;
                                q_col_arr[q_back] = next_c;
                                q_back++;
                            }
                        }
                    }
                }
            }
        }

        int optimal_tile_r = -1, optimal_tile_c = -1;
        int lowest_cost = 2e9;
    }
}

```

```

        for (int i = 0; i < board_size; ++i) {
            for (int j = 0; j < board_size; ++j) {
                if (!is_fixed[i][j] && current_grid[i][j] == expected_color && path_cost[i][j] != 1e9) {
                    int fetch_distance = abs(blank_row - i) + abs(blank_col - j);
                    int rng_factor = rand() % 9;

                    int total_eval_cost = (path_cost[i][j] * 5) + fetch_distance + rng_factor;
                    if (total_eval_cost < lowest_cost) {
                        lowest_cost = total_eval_cost;
                        optimal_tile_r = i;
                        optimal_tile_c = j;
                    }
                }
            }
        }

        int track_r = optimal_tile_r, track_c = optimal_tile_c;
        while (track_r != target_r || track_c != target_c) {
            pair<int, int> next_hop = calculate_tile_step(track_r, track_c, target_r, target_c);
            if (next_hop.first == -1) {
                is_deadlocked = true;
                break;
            }

            int n_r = next_hop.first, n_c = next_hop.second;

            string setup_move = route_blank_slot(n_r, n_c, track_r, track_c);
            accumulated_moves += setup_move;
            apply_actions(setup_move);

            string shift_move = route_blank_slot(track_r, track_c, -1, -1);
            accumulated_moves += shift_move;
            apply_actions(shift_move);
            apply_actions(shift_move);

            track_r = n_r;
            track_c = n_c;
        }
        if (is_deadlocked) break;
        is_fixed[target_r][target_c] = true;
    }
    if (is_deadlocked) break;

    if (!is_deadlocked) {
        accumulated_moves = clean_up_redundant_moves(accumulated_moves);
        int final_sequence_len = accumulated_moves.length();

        if (final_sequence_len < global_minimum_moves) {
            global_minimum_moves = final_sequence_len;
            cout << "[Attempt " << attempt << "] New Best Moves : " << global_minimum_moves << endl;

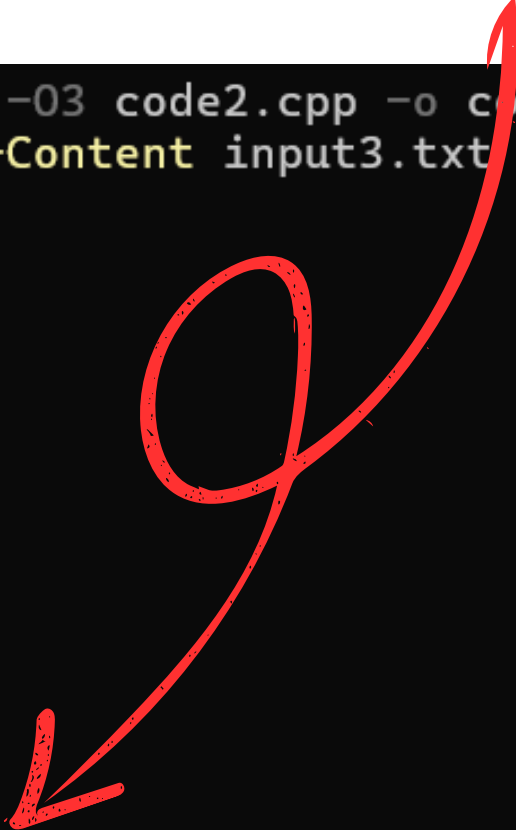
            ofstream output_file("resP2_1.txt");
            output_file << accumulated_moves << "s\n";
            output_file.close();
        } else if (attempt % 1000 == 0) {
            cout << "(Progress Update) Attempt " << attempt << " | Current Best : " << global_minimum_moves << endl;
        }
    }

    return 0;
}

```

Greedy BFS + State Space Search + Monte Carlo Method

```
PS D:\CP_2nd_Year\Algo\FimGraderAlgo\FilmRubik\video> g++ -O3 code2.cpp -o code2
PS D:\CP_2nd_Year\Algo\FimGraderAlgo\FilmRubik\video> Get-Content input3.txt | .\code2.exe
=== Start Optimization (Monte Carlo) ===
[Attempt 1] New Best Moves : 23089
[Attempt 2] New Best Moves : 21893
[Attempt 4] New Best Moves : 21643
[Attempt 13] New Best Moves : 21277
[Attempt 37] New Best Moves : 18321
(Progress Update) Attempt 1000 | Current Best : 18321
[Attempt 1377] New Best Moves : 17793
(Progress Update) Attempt 2000 | Current Best : 17793
[Attempt 2255] New Best Moves : 16773
(Progress Update) Attempt 3000 | Current Best : 16773
[Attempt 3682] New Best Moves : 16771
(Progress Update) Attempt 4000 | Current Best : 16771
[Attempt 4046] New Best Moves : 16609
(Progress Update) Attempt 5000 | Current Best : 16609
(Progress Update) Attempt 6000 | Current Best : 16609
(Progress Update) Attempt 7000 | Current Best : 16609
(Progress Update) Attempt 8000 | Current Best : 16609
(Progress Update) Attempt 9000 | Current Best : 16609
(Progress Update) Attempt 10000 | Current Best : 16609
(Progress Update) Attempt 11000 | Current Best : 16609
(Progress Update) Attempt 12000 | Current Best : 16609
[Attempt 12563] New Best Moves : 16189
(Progress Update) Attempt 13000 | Current Best : 16189
```



```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <cmath>
5  #include <algorithm>
6  #include <fstream>
7  #include <ctime>
8  #include <cstdlib>
9
10 using namespace std;
11
12 const int MAX_GRID = 75;
13 int initial_grid[MAX_GRID][MAX_GRID];
14 int current_grid[MAX_GRID][MAX_GRID];
15 int goal_pattern[MAX_GRID][MAX_GRID];
16 bool is_fixed[MAX_GRID][MAX_GRID];
17 int board_size, init_blank_row, init_blank_col, blank_row, blank_col;
18
19 const int d_row[] = {-1, 1, 0, 0};
20 const int d_col[] = {0, 0, -1, 1};
21 const char action_labels[] = {'D', 'U', 'R', 'L'};
22
23 int bfs_visited[MAX_GRID][MAX_GRID];
24 int current_bfs_ticket = 0;
25 int trace_r[MAX_GRID][MAX_GRID];
26 int trace_c[MAX_GRID][MAX_GRID];
27 char trace_move[MAX_GRID][MAX_GRID];
28

```

```

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
    srand(time(NULL));

    cin >> board_size;

    for (int i = 0; i < board_size; ++i) {
        for (int j = 0; j < board_size; ++j) {
            cin >> initial_grid[i][j];
            if (initial_grid[i][j] == -1) {
                init_blank_row = i;
                init_blank_col = j;
            }
        }
    }

    for (int i = 0; i < board_size - 2; ++i) {
        for (int j = 0; j < board_size - 2; ++j) {
            cin >> goal_pattern[i][j];
        }
    }

    int global_minimum_moves = 2e9;

    cout << "=== Start Otpimization (Monte Carlo) ===" << endl;
}

```

```

cout << "=== Start Otpimization (Monte Carlo) ===" << endl;

for (int attempt = 1; ; ++attempt) {
    restore_board_state();
    string accumulated_moves = "";
    int path_cost[MAX_GRID][MAX_GRID];
    bool is_deadlocked = false;

    for (int target_r = 1; target_r <= board_size - 2; ++target_r) {
        for (int step = 1; step <= board_size - 2; ++step) {
            int target_c = (target_r % 2 != 0) ? step : (board_size - 1 - step);
            int expected_color = goal_pattern[target_r - 1][target_c - 1];

```

6	1	5
6	3	2
4	1	2



1	2	3	4	5
2	6	1	3	4
-1	6	3	2	5
5	4	1	2	1
6	5	4	3	6

⋮
⋮ main algorithm
⋮
⋮

```

    if (!is_deadlocked) {
        accumulated_moves = clean_up_redundant_moves(accumulated_moves);
        int final_sequence_len = accumulated_moves.length();

        if (final_sequence_len < global_minimum_moves) {
            global_minimum_moves = final_sequence_len;
            cout << "[Attempt " << attempt << "] New Best Moves : " << global_minimum_moves << endl;

            ofstream output_file("result.txt");
            output_file << accumulated_moves << "S\n";
            output_file.close();
        } else if (attempt % 1000 == 0) {
            cout << "(Progress Update) Attempt " << attempt << " | Current Best : " << global_minimum_moves << endl;
        }
    }
}

return 0;
}

```

```

void restore_board_state() {
    for (int i = 0; i < board_size; ++i) {
        for (int j = 0; j < board_size; ++j) {
            current_grid[i][j] = initial_grid[i][j];
            is_fixed[i][j] = false;
        }
    }
    blank_row = init_blank_row;
    blank_col = init_blank_col;
}

string clean_up_redundant_moves(string raw_moves) {
    bool is_modified = true;
    while (is_modified) {
        is_modified = false;
        string optimized = "";
        for (char action : raw_moves) {
            if (!optimized.empty()) {
                char prev_action = optimized.back();
                if ((prev_action == 'U' && action == 'D') || (prev_action == 'D' && action == 'U') ||
                    (prev_action == 'L' && action == 'R') || (prev_action == 'R' && action == 'L')) {
                    optimized.pop_back();
                    is_modified = true;
                    continue;
                }
            }
            optimized.push_back(action);
        }
        if (raw_moves.length() != optimized.length()) {
            raw_moves = optimized;
            is_modified = true;
        }
    }
    return raw_moves;
}

```

```

pair<int, int> calculate_tile_step(int start_r, int start_c, int end_r, int end_c) {
    current_bfs_ticket++;
    int q_front = 0, q_back = 0;
    int q_row_arr[10000], q_col_arr[10000];

    q_row_arr[q_back] = start_r;
    q_col_arr[q_back] = start_c;
    q_back++;
    bfs_visited[start_r][start_c] = current_bfs_ticket;

    bool target_reached = false;

    while (q_front < q_back) {
        int r = q_row_arr[q_front], c = q_col_arr[q_front];
        q_front++;

        if (r == end_r && c == end_c) {
            target_reached = true;
            break;
        }

        for (int i = 0; i < 4; ++i) {
            int next_r = r + d_row[i], next_c = c + d_col[i];
            if (next_r >= 0 && next_r < board_size && next_c >= 0 && next_c < board_size) {
                if (is_fixed[next_r][next_c]) continue;
                if (bfs_visited[next_r][next_c] != current_bfs_ticket) {
                    bfs_visited[next_r][next_c] = current_bfs_ticket;
                    trace_r[next_r][next_c] = r;
                    trace_c[next_r][next_c] = c;
                    q_row_arr[q_back] = next_r;
                    q_col_arr[q_back] = next_c;
                    q_back++;
                }
            }
        }
    }

    int temp_r = end_r, temp_c = end_c;
    while (trace_r[temp_r][temp_c] != start_r || trace_c[temp_r][temp_c] != start_c) {
        int pr = trace_r[temp_r][temp_c];
        int pc = trace_c[temp_r][temp_c];
        temp_r = pr;
        temp_c = pc;
    }
    return {temp_r, temp_c};
}

```

6	1	5
6	3	2
4	1	2



1	2	3	4	5
2	6	1	3	4
-1	6	3	2	5
5	4	1	2	1
6	5	4	3	6

```

string route_blank_slot(int dest_r, int dest_c, int block_r, int block_c) {
    if (blank_row == dest_r && blank_col == dest_c) return "";

    current_bfs_ticket++;
    int q_front = 0, q_back = 0;
    int q_row_arr[10000], q_col_arr[10000];

    q_row_arr[q_back] = blank_row;
    q_col_arr[q_back] = blank_col;
    q_back++;
    bfs_visited[blank_row][blank_col] = current_bfs_ticket;

    bool target_reached = false;

    while (q_front < q_back) {
        int r = q_row_arr[q_front], c = q_col_arr[q_front];
        q_front++;

        if (r == dest_r && c == dest_c) {
            target_reached = true;
            break;
        }

        for (int i = 0; i < 4; ++i) {
            int next_r = r + d_row[i], next_c = c + d_col[i];
            if (next_r >= 0 && next_r < board_size && next_c >= 0 && next_c < board_size) {
                if (is_fixed[next_r][next_c]) continue;
                if (next_r == block_r && next_c == block_c) continue;

                if (bfs_visited[next_r][next_c] != current_bfs_ticket) {
                    bfs_visited[next_r][next_c] = current_bfs_ticket;
                    trace_r[next_r][next_c] = r;
                    trace_c[next_r][next_c] = c;
                    trace_move[next_r][next_c] = action_labels[i];
                    q_row_arr[q_back] = next_r;
                    q_col_arr[q_back] = next_c;
                    q_back++;
                }
            }
        }
    }

    string final_path = "";
    if (!target_reached) return final_path;

    int temp_r = dest_r, temp_c = dest_c;
    while (temp_r != blank_row || temp_c != blank_col) {
        final_path += trace_move[temp_r][temp_c];
        int pr = trace_r[temp_r][temp_c];
        int pc = trace_c[temp_r][temp_c];
        temp_r = pr;
        temp_c = pc;
    }
    reverse(final_path.begin(), final_path.end());
    return final_path;
}

```

6	1	5
6	3	2
4	1	2



1	2	3	4	5
2	6	1	3	4
-	1	6	3	2
5	4	1	2	1
6	5	4	3	6

```
✓ void apply_actions(const string& sequence) {  
✓     for (char move : sequence) {  
✓         int direction_idx = -1;  
         for (int i = 0; i < 4; ++i) {  
             if (action_labels[i] == move) direction_idx = i;  
         }  
         int next_r = blank_row + d_row[direction_idx];  
         int next_c = blank_col + d_col[direction_idx];  
         swap(current_grid[blank_row][blank_col], current_grid[next_r][next_c]);  
         blank_row = next_r;  
         blank_col = next_c;  
     }  
}
```

```

for (int i = 0; i < board_size; ++i) {
    for (int j = 0; j < board_size; ++j) {
        path_cost[i][j] = 1e9;
    }
}

current_bfs_ticket++;
int q_front = 0, q_back = 0;
int q_row_arr[10000], q_col_arr[10000];

q_row_arr[q_back] = target_r;
q_col_arr[q_back] = target_c;
q_back++;

path_cost[target_r][target_c] = 0;
bfs_visited[target_r][target_c] = current_bfs_ticket;

while (q_front < q_back) {
    int curr_r = q_row_arr[q_front], curr_c = q_col_arr[q_front];
    q_front++;
    for (int i = 0; i < 4; ++i) {
        int next_r = curr_r + d_row[i], next_c = curr_c + d_col[i];
        if (next_r >= 0 && next_r < board_size && next_c >= 0 && next_c < board_size) {
            if (!is_fixed[next_r][next_c] && bfs_visited[next_r][next_c] != current_bfs_ticket) {
                bfs_visited[next_r][next_c] = current_bfs_ticket;
                path_cost[next_r][next_c] = path_cost[curr_r][curr_c] + 1;
                q_row_arr[q_back] = next_r;
                q_col_arr[q_back] = next_c;
                q_back++;
            }
        }
    }
}

int optimal_tile_r = -1, optimal_tile_c = -1;
int lowest_cost = 2e9;

```

```
for (int i = 0; i < board_size; ++i) {  
    for (int j = 0; j < board_size; ++j) {  
        if (!is_fixed[i][j] && current_grid[i][j] == expected_color && path_cost[i][j] != 1e9) {  
            int fetch_distance = abs(blank_row - i) + abs(blank_col - j);  
            int rng_factor = rand() % 9;  
  
            int total_eval_cost = (path_cost[i][j] * 5) + fetch_distance + rng_factor;  
            if (total_eval_cost < lowest_cost) {  
                lowest_cost = total_eval_cost;  
                optimal_tile_r = i;  
                optimal_tile_c = j;  
            }  
        }  
    }  
}
```

```

int track_r = optimal_tile_r, track_c = optimal_tile_c;
while (track_r != target_r || track_c != target_c) {
    pair<int, int> next_hop = calculate_tile_step(track_r, track_c, target_r, target_c);
    if (next_hop.first == -1) {
        is_deadlocked = true;
        break;
    }

    int n_r = next_hop.first, n_c = next_hop.second;

    string setup_move = route_blank_slot(n_r, n_c, track_r, track_c);
    accumulated_moves += setup_move;
    apply_actions(setup_move);

    string shift_move = route_blank_slot(track_r, track_c, -1, -1);
    accumulated_moves += shift_move;
    apply_actions(shift_move);

    track_r = n_r;
    track_c = n_c;
}

if (is_deadlocked) break;
is_fixed[target_r][target_c] = true;
}
if (is_deadlocked) break;
}

```